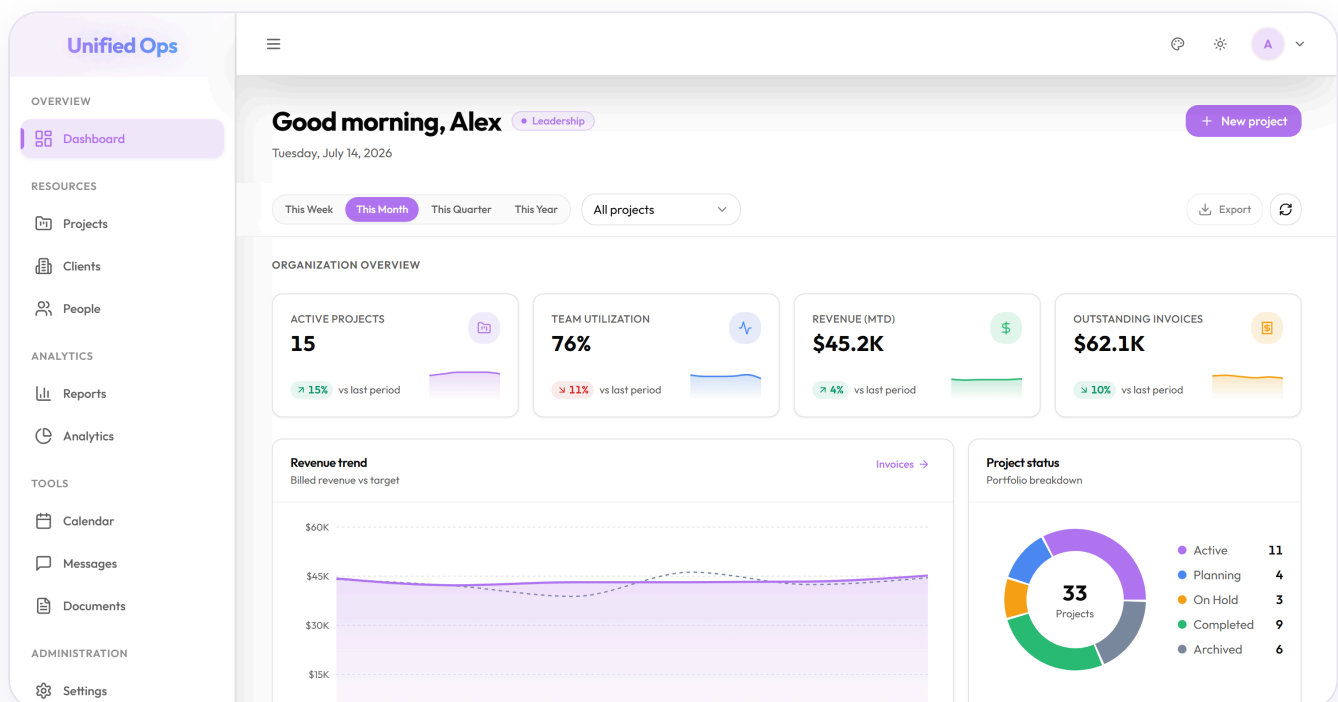


CLIENT-FACING CASE STUDY · CHALLENGE → SOLUTION → RESULT

Running the back office of a professional-services firm on one system

A multi-tenant SaaS that unifies clients, projects, people, time, approvals, billing, and documents — with a strict workspace boundary and server-enforced access control at every step.



Multi-tenant SaaS

FastAPI · PostgreSQL

Next.js · TypeScript

RBAC · JWT

S3-compatible storage

Professional-services firms deliver client work through a patchwork of disconnected tools — and pay for it in re-keyed data, guessed-at margins, and risk.

An agency, consultancy, or software studio typically runs one app for time tracking, another for project management, a spreadsheet for capacity planning, a separate system for invoicing, and email for everything in between. Nothing shares a data model, so the same facts are entered two or three times, and the numbers that matter — utilization, project margin, what's billable — are reconstructed by hand at month-end instead of being available on demand.

- **Fragmented operations**

Time, projects, capacity, and billing live in separate silos. Work is re-keyed between them, invoicing lags delivery by weeks, and margin is a guess until the books close.

- **Access control sprawl**

Every tool is its own island of permissions and its own place for client data to sit. For a firm juggling many engagements, the risk surface grows with every new app.

- **No single source of truth**

Because the pipeline isn't connected, leadership can't see delivery, staffing, and billing in one place — decisions wait on someone assembling a spreadsheet.

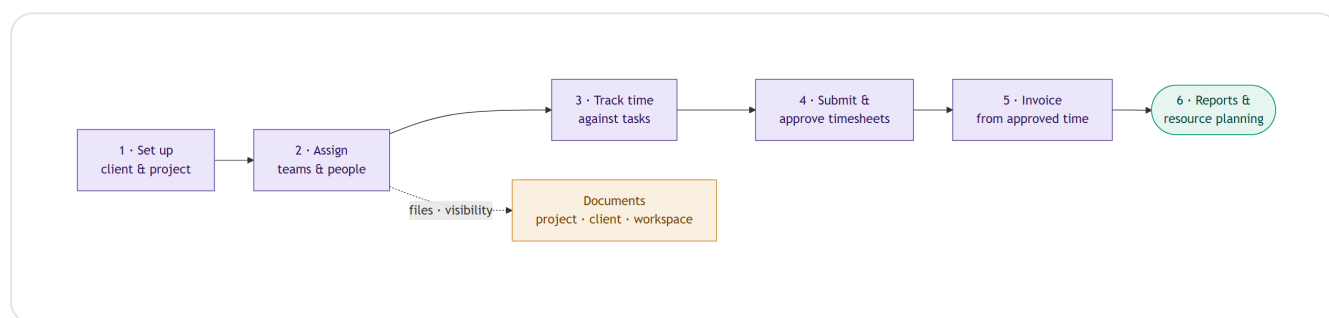
- **Multi-client data risk**

A services firm holds many clients' data at once. Without a hard tenancy boundary, keeping one client's information from ever reaching another is a matter of discipline, not design.

The goal: collapse the stack into one integrated system where a team plans work, staffs it, tracks time against it, approves it, bills it, and stores the paperwork — without leaving the product, and without any workspace's data ever touching another's.

One multi-tenant platform for the whole delivery lifecycle, built on a clean, layered architecture with tenant isolation designed in — not bolted on.

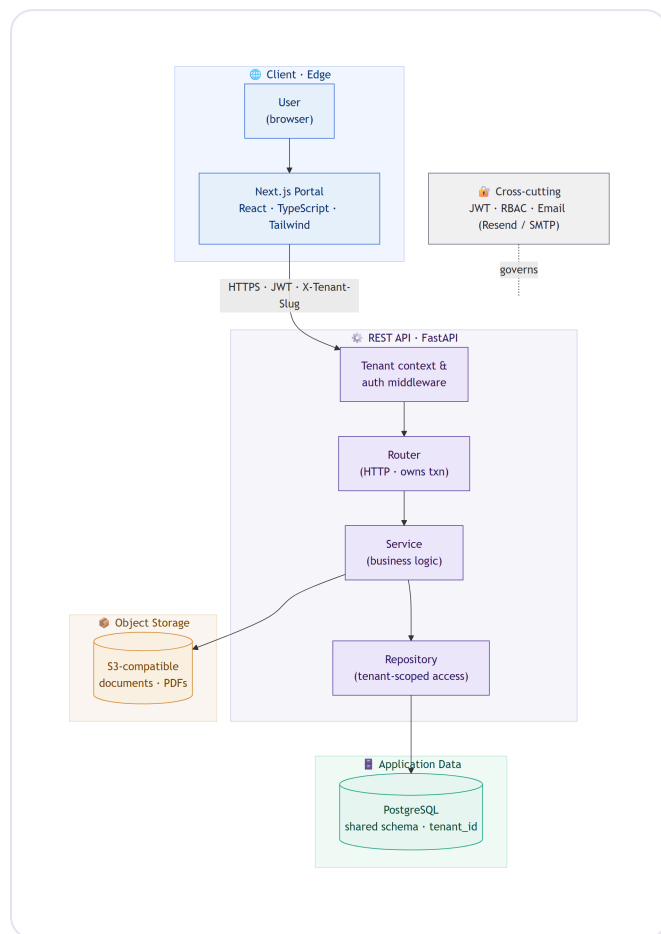
The product is organized around how a services team actually works: set up a client and a project, staff it with functional teams and people, track time against tasks, submit and approve timesheets, invoice from the approved time, and report on utilization and resource planning — with documents scoped to a project, a client, or the whole workspace.



The end-to-end delivery lifecycle — a single connected pipeline, no re-keying between stages.

Layered, multi-tenant architecture

A shared database with a **tenant_id** discriminator on every domain table. The workspace is derived from the request's subdomain and carried as a header; the API cross-checks it against the caller's signed token and active membership on *every* request, and data access is auto-scoped to the tenant at the repository layer — so cross-tenant reads are structurally impossible rather than merely discouraged.

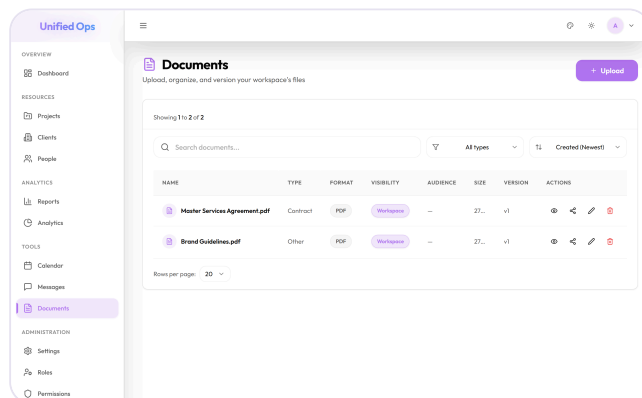
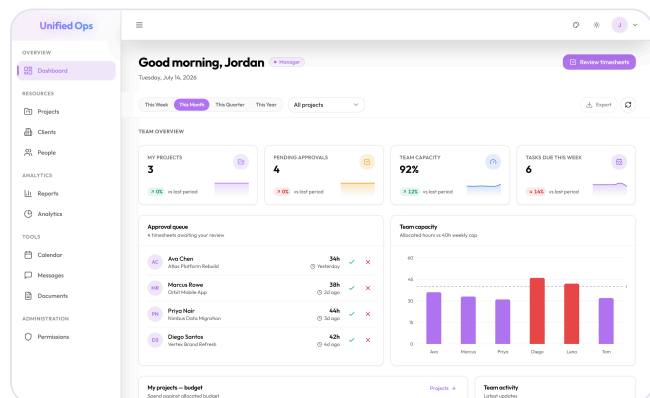


A workspace-aware Next.js portal over a layered FastAPI service.

How it's built

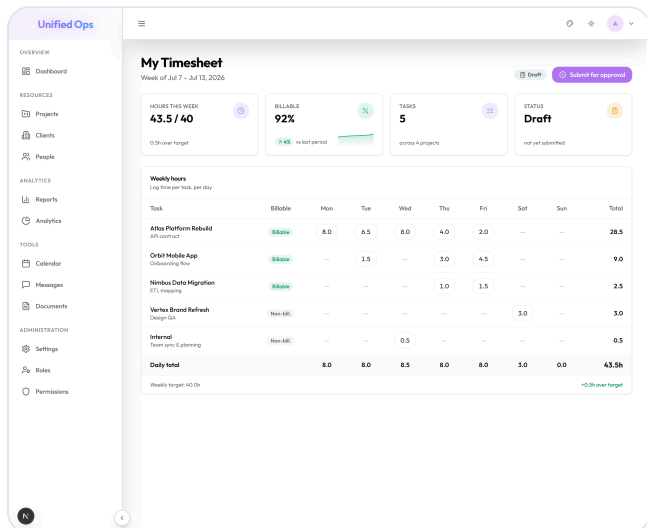
- **Thin routers, real services.** HTTP routers own the transaction boundary; framework-agnostic services hold all business logic; repositories own tenant-scoped, soft-delete-aware data access.
- **Server-enforced RBAC.** Fine-grained resource:action permissions checked on every endpoint; the UI shows what a user *may* do, the server decides what they *can*.
- **Role-aware UI.** The same dashboard renders leadership, manager, or member views resolved from the signed-in user's permissions.

Application data lives in PostgreSQL (shared schema, **tenant_id** discriminator); documents and generated PDFs live in S3-compatible object storage.

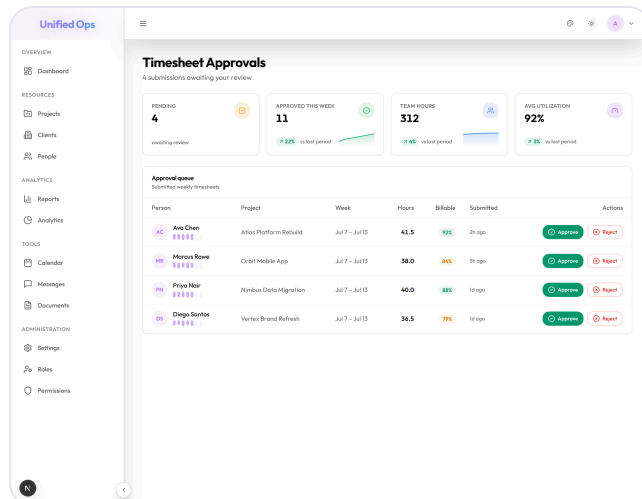


Role-aware dashboards and workspace documents with per-file visibility.

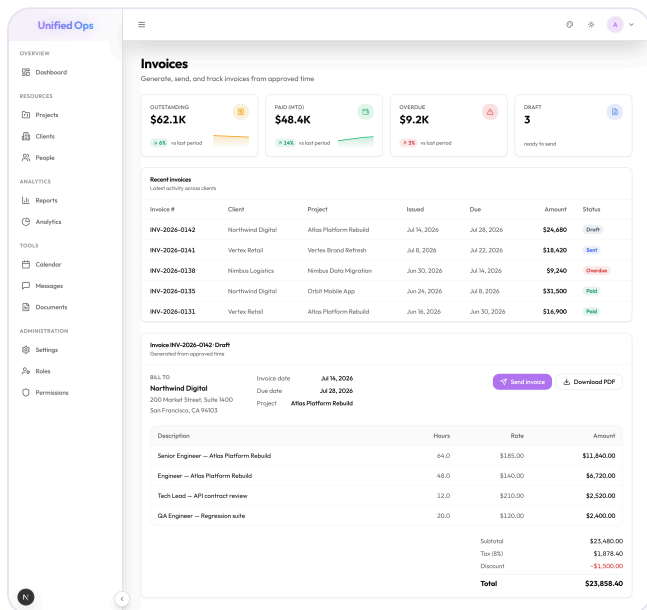
The same connected pipeline, screen by screen — from logging time to sending the invoice and reading the margin.



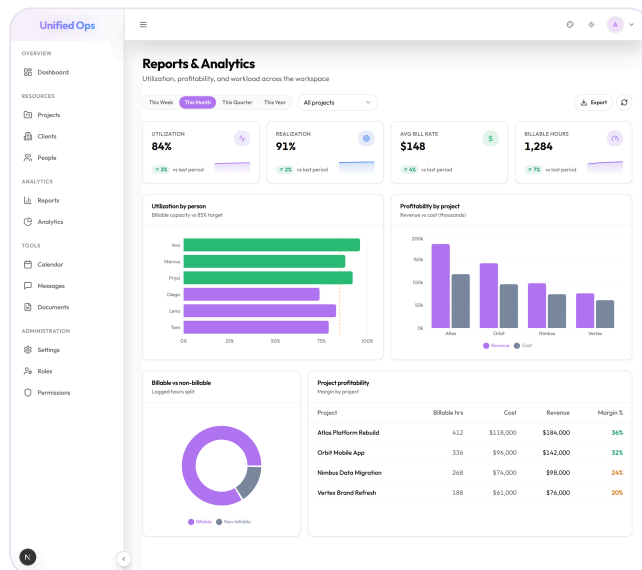
Track — a weekly timesheet: hours per task, billable flags, submit for approval.



Approve — a manager reviews and signs off submitted time in one place.



Bill — an invoice generated from approved time, with tax and lifecycle status.



Report — utilization vs. target, project profitability, and margin.

A production-grade platform whose correctness and multi-tenant safety are demonstrable — proven by tests, not just asserted in prose.

4

RBAC role tiers plus fully custom roles, enforced server-side

0

Cross-tenant reads — foreign-workspace access returns *not found*, no existence leak

5

Quality gates on every change: format · lint · type · security · tests

- **Tenant isolation you can prove.** Auto-scoped data access plus a dedicated cross-tenant test suite: foreign-workspace reads 404, workspace/identity mismatches are rejected, and a revoked member's still-valid token is locked out immediately.
- **Security-first access control.** Every authenticated endpoint declares the permission it requires, and privilege-escalation guards stop a user granting access beyond their own.
- **Correctness under concurrency.** Money- and ownership-sensitive flows — billing rates, the last remaining owner of a workspace, uniqueness constraints — are guarded with explicit invariants and race-safe write paths.
- **Depth of automated testing.** Layered unit, integration, and end-to-end suites cover happy paths, negative cases, boundaries, multi-tenant isolation, RBAC matrices, and data integrity; browser tests drive the real UI against a live backend.
- **Breadth of domain.** Identity & workspaces, clients & contacts, projects, functional teams & people, time & approvals, invoicing, resource planning, reporting, documents, and issue-tracker / calendar integrations.

Backend	Python · FastAPI · Pydantic v2 · SQLAlchemy 2.0 · Alembic · PostgreSQL (UUIDv7, soft deletes)
Frontend	Next.js · React · TypeScript · Tailwind CSS · shadcn/ui
Platform	JWT (HS256) auth · Argon2 hashing · S3-compatible storage · pluggable email (SMTP / transactional API)
Delivery	Dockerized services · CI quality gates · automated database migrations

What's next — Document Intelligence

A permission-aware, RAG-based assistant that lets any stakeholder ask plain-language questions about company policy, project requirements, or delivered solutions — with retrieval filtered by the same tenancy and document-visibility rules the platform already enforces, so answers never cross a boundary.